EECS 348 Assignment 3
GenAI Analysis

1. GenAI Programs

GenAI Program 1: ChatGPT
ChatGPT through the ChatGPT website using a web browser.
GenAI Program 2: Google Gemini
I accessed Google Gemini through the Gemini website using a web browser.
2. Prompt Used
Write a beginner-friendly C++ program.
The program will prioritize emails for a busy company CEO.
Here are the requirements:
- Use objects, not standalone functions.
- Implement a MaxHeap from scratch.
- The MaxHeap must use a list-based implementation.
- Do not use any pre-existing heap or priority queue modules.
- The MaxHeap will implement the priority queue for the CEO's emails.
Input format:
EMAIL <sender category>,<subject line>,<date>
Sender categories and priority:
Boss = highest priority
Subordinate = next
Peer = next
ImportantPerson = next
OtherPerson = lowest
If there is more than one email from the same sender category, the newest email should be read first.
The subject line may contain spaces but not commas.
The date format is MM-DD-YYYY.
Commands:
NEXT = display the next highest-priority unread email:
Sender:
Subject:
Date:

READ = remove the highest-priority email from the queue without displaying it.

COUNT = display the current number of unread emails.

The program must correctly handle:
- NEXT when there are no emails
- READ when there are no emails
- two NEXT commands in a row without READ; both NEXT commands should display the same email
- two READ commands in a row without NEXT; each READ should remove the highest-priority email

Use the following sample input:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

The expected output for the sample should be:

```
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.
```

3. ChatGPT Generated Code

```
#include <iostream>

#include <vector>

#include <string>

#include <sstream>



using namespace std;



// Represents one email

class Email {
```

```
public:
    string sender;
    string subject;
    string date;
    int priority;

    Email(string s, string sub, string d) {
        sender = s;
        subject = sub;
        date = d;

        // Assign priority based on sender category
        if (sender == "Boss") {
            priority = 5;
        }
        else if (sender == "Subordinate") {
            priority = 4;
        }
        else if (sender == "Peer") {
            priority = 3;
        }
        else if (sender == "ImportantPerson") {
            priority = 2;
        }
        else {
```

```
            priority = 1;
        }
    }
};

// MaxHeap class
class MaxHeap {
private:
    vector<Email> heap;

    // Returns true if email a should come before email b
    bool higherPriority(Email a, Email b) {
        if (a.priority != b.priority) {
            return a.priority > b.priority;
        }

        // If same sender category, newer date comes first
        return newerDate(a.date, b.date);
    }

    // Converts MM-DD-YYYY into a number that can be compared
    int dateValue(string date) {
        int month, day, year;
        char dash1, dash2;
```

```
    stringstream ss(date);
    ss >> month >> dash1 >> day >> dash2 >> year;

    return year * 10000 + month * 100 + day;
}

bool newerDate(string date1, string date2) {
    return dateValue(date1) > dateValue(date2);
}

// Moves an item upward in the heap
void heapifyUp(int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;

        if (higherPriority(heap[index], heap[parent])) {
            swap(heap[index], heap[parent]);
            index = parent;
        }
        else {
            break;
        }
    }
}
```

```
// Moves an item downward in the heap
void heapifyDown(int index) {
    int size = heap.size();

    while (true) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < size &&
            higherPriority(heap[left], heap[largest])) {
            largest = left;
        }

        if (right < size &&
            higherPriority(heap[right], heap[largest])) {
            largest = right;
        }

        if (largest != index) {
            swap(heap[index], heap[largest]);
            index = largest;
        }
        else {
            break;
```

```
            }
        }
    }

public:
    // Add an email to the heap
    void insert(Email email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    // Return the highest-priority email without removing it
    Email getMax() {
        return heap[0];
    }

    // Remove the highest-priority email
    void removeMax() {
        if (heap.empty()) {
            return;
        }

        heap[0] = heap.back();
        heap.pop_back();
```

```
        if (!heap.empty()) {
            heapifyDown(0);
        }
    }

    // Return the number of emails
    int size() {
        return heap.size();
    }

    bool empty() {
        return heap.empty();
    }
};

// Main program
class EmailSystem {
private:
    MaxHeap emails;

    // Turns an EMAIL line into an Email object
    Email createEmail(string line) {
        // Remove "EMAIL "
        line = line.substr(6);
```

```
        string sender;
        string subject;
        string date;

        size_t comma1 = line.find(',');
        size_t comma2 = line.find(',', comma1 + 1);

        sender = line.substr(0, comma1);
        subject = line.substr(comma1 + 1, comma2 - comma1 - 1);
        date = line.substr(comma2 + 1);

        return Email(sender, subject, date);
    }

public:
    void processCommand(string line) {
        if (line.substr(0, 6) == "EMAIL ") {
            Email email = createEmail(line);
            emails.insert(email);
        }

        else if (line == "COUNT") {
            cout << "There are " << emails.size()
                 << " emails to read." << endl;
        }
```

```
        else if (line == "NEXT") {
            if (emails.empty()) {
                cout << "There are no emails to read." << endl;
            }
            else {
                Email email = emails.getMax();

                cout << "Next email:" << endl;
                cout << "Sender: " << email.sender << endl;
                cout << "Subject: " << email.subject << endl;
                cout << "Date: " << email.date << endl;
            }
        }

        else if (line == "READ") {
            if (!emails.empty()) {
                emails.removeMax();
            }
        }
    }
};

int main() {
    EmailSystem system;
```

```
    string line;

    while (getline(cin, line)) {
        system.processCommand(line);
    }

    return 0;
}
```

4. Google Gemini Generated Code

```
#include <iostream>
#include <string>
#include <sstream>
#include <vector>

// Structure representing an Email object
struct Email {
    std::string senderCategory;
    std::string subject;
    std::string date;

    // Helper to assign a numerical priority to each category
    int getCategoryPriority() const {
        if (senderCategory == "Boss") return 5;
```

```
    if (senderCategory == "Subordinate") return 4;
    if (senderCategory == "Peer") return 3;
    if (senderCategory == "ImportantPerson") return 2;
    return 1; // "OtherPerson" or any unknown category
}

// Helper to convert date "MM-DD-YYYY" to "YYYYMMDD" integer for easy comparison
int getDateKey() const {
    if (date.length() != 10) return 0;
    int month = std::stoi(date.substr(0, 2));
    int day   = std::stoi(date.substr(3, 2));
    int year  = std::stoi(date.substr(6, 4));
    return year * 10000 + month * 100 + day;
}

// Greater-than operator overloaded to define priority ordering
bool operator>(const Email& other) const {
    int p1 = getCategoryPriority();
    int p2 = other.getCategoryPriority();

    if (p1 != p2) {
        return p1 > p2;
    }
    // Tie-breaker: Newer date gets higher priority
    return getDateKey() > other.getDateKey();
```

```
    }
};

// MaxHeap implemented from scratch using dynamic array/list storage (std::vector)
class MaxHeap {
private:
    std::vector<Email> heap;

    // Percolate UP: Restore heap property after inserting a new element
    void heapifyUp(size_t index) {
        while (index > 0) {
            size_t parent = (index - 1) / 2;
            if (heap[index] > heap[parent]) {
                std::swap(heap[index], heap[parent]);
                index = parent;
            } else {
                break;
            }
        }
    }

    // Percolate DOWN: Restore heap property after extracting the root element
    void heapifyDown(size_t index) {
        size_t size = heap.size();
        while (true) {
```

```
        size_t largest = index;
        size_t leftChild = 2 * index + 1;
        size_t rightChild = 2 * index + 2;

        if (leftChild < size && heap[leftChild] > heap[largest]) {
            largest = leftChild;
        }
        if (rightChild < size && heap[rightChild] > heap[largest]) {
            largest = rightChild;
        }

        if (largest != index) {
            std::swap(heap[index], heap[largest]);
            index = largest;
        } else {
            break;
        }
    }
}

public:
    // Insert an email into the priority queue
    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
```

```
}

// Peak at the highest-priority email without removing it
bool peek(Email& result) const {
    if (heap.empty()) {
        return false;
    }
    result = heap[0];
    return true;
}

// Remove the highest-priority email from the heap
bool extractMax() {
    if (heap.empty()) {
        return false;
    }
    heap[0] = heap.back();
    heap.pop_back();
    if (!heap.empty()) {
        heapifyDown(0);
    }
    return true;
}

// Return current count of items in the heap
```

```
    size_t size() const {
        return heap.size();
    }

    bool empty() const {
        return heap.empty();
    }
};

// Class to manage CEO Email Operations
class EmailManager {
private:
    MaxHeap priorityQueue;

public:
    // Parse input string: EMAIL Sender,Subject,Date
    void processEmailCommand(const std::string& line) {
        // Skip "EMAIL " prefix
        std::string data = line.substr(6);
        std::stringstream ss(data);

        Email email;
        std::getline(ss, email.senderCategory, ',');
        std::getline(ss, email.subject, ',');
        std::getline(ss, email.date);
```

```
    priorityQueue.insert(email);
}

// Display the next highest priority email without deleting it
void showNext() const {
    Email topEmail;
    if (priorityQueue.peek(topEmail)) {
        std::cout << "Next email:\n";
        std::cout << "Sender: " << topEmail.senderCategory << "\n";
        std::cout << "Subject: " << topEmail.subject << "\n";
        std::cout << "Date: " << topEmail.date << "\n";
    } else {
        std::cout << "No emails to read.\n";
    }
}

// Remove the top email without displaying
void readEmail() {
    priorityQueue.extractMax();
}

// Print total count of unread emails
void showCount() const {
    std::cout << "There are " << priorityQueue.size() << " emails to read.\n";
```

```
    }
};

int main() {
    EmailManager manager;
    std::string line;

    // Read commands line by line from standard input
    while (std::getline(std::cin, line)) {
        if (line.empty()) continue;

        if (line.rfind("EMAIL ", 0) == 0) {
            manager.processEmailCommand(line);
        } else if (line == "NEXT") {
            manager.showNext();
        } else if (line == "READ") {
            manager.readEmail();
        } else if (line == "COUNT") {
            manager.showCount();
        }
    }

    return 0;
}
```

5. Code Analysis

ChatGPT

Correctness: The ChatGPT program correctly implements a list-based MaxHeap and uses it as a priority queue. It correctly prioritizes emails by sender category and uses the newer date when two emails have the same sender category. The program was tested using the sample test file provided in the assignment.

Execution Time: The MaxHeap insertion operation takes O(log n) time in the worst case because an email may move upward through the heap. Removing the highest-priority email also takes O(log n) time because the replacement element may move downward. Viewing the highest-priority email takes O(1) time.

Space Complexity: The program stores the unread emails in the heap, so the space complexity is O(n), where n is the number of unread emails.

Maintainability: The program separates the Email, MaxHeap, and EmailSystem responsibilities into classes. This makes the code easier to understand and modify.

Gemini

Correctness: The Gemini program correctly implements the required MaxHeap and priority queue. It correctly handles the sender priority categories and uses the newer date as the tiebreaker. The program was tested using the sample test file provided in the assignment.

Execution Time: The MaxHeap insertion and removal operations take O(log n) time in the worst case, while viewing the highest-priority email takes O(1) time.

Space Complexity: The heap stores each unread email once, resulting in O(n) space complexity.

Maintainability: The Gemini program separates the Email, MaxHeap, and EmailManager responsibilities into classes. It also uses a comparison operator and safe heap operations, which makes the implementation easier to read and maintain.

6. Selected Program: I selected the Gemini-generated program as the starting point for my final implementation. Both programs successfully handled the provided sample test. I selected Gemini because its heap interface safely checks for an empty heap, it uses const methods where appropriate, and its comparison operator makes the priority comparison easier to read.

7. Improvements that i made

Correctness: I tested the program using the sample test file and verified that emails are prioritized according to the required sender categories and that newer emails are prioritized when sender categories are the same.

Execution Time: I kept the MaxHeap implementation because insertion and removal take O(log n) time in the worst case, while accessing the highest-priority email takes O(1) time.

Space Complexity: I kept the heap as the primary storage structure so that each unread email is stored once. The resulting space complexity is O(n).

Maintainability:I organized the program into separate Email, MaxHeap, and EmailManager classes. I also added program documentation and comments explaining the major sections of the code.